In [ ]:
```python
import json
import marimo as mo
import numpy as np
import optuna
import plotly.graph_objects as go
import polars as pl
import xgboost as xgb
from pathlib import Path
from sklearn.calibration import CalibratedClassifierCV, calibration_curve
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    log_loss,
    roc_auc_score,
)
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import StandardScaler

optuna.logging.set_verbosity(optuna.logging.WARNING)
```

1. Data loading and preparation

In [ ]:
```python
PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = PROJECT_ROOT / "data" / "processed" / "loans_processed.parquet"

df = pl.read_parquet(DATA_PATH)

TARGET = "default_21d"
NON_FEATURE_COLUMNS = ["loan_id", "loan_issue_date", "default_14d", TARGET]
CATEGORICAL_FEATURES = ["merchant_group", "merchant_category"]

feature_df = df.drop(NON_FEATURE_COLUMNS)
numeric_feature_df = feature_df.drop(CATEGORICAL_FEATURES)
y = df[TARGET].to_numpy().astype(int)
```

> **Note on temporal trends:** EDA showed a decreasing default rate over the observation window. However, the time span is too short to distinguish a real trend from seasonality or noise, so we treat the data as i.i.d. tabular and use a random split.

2. Train-test split

In [ ]:
```python
X_full_pd = feature_df.to_pandas()
for _col in CATEGORICAL_FEATURES:
    X_full_pd[_col] = X_full_pd[_col].astype("category")

X_numeric = numeric_feature_df.to_numpy().astype(np.float64)
```

```
numeric_columns = numeric_feature_df.columns

indices = np.arange(len(y))
train_idx, test_idx = train_test_split(
    indices, test_size=0.2, random_state=42, stratify=y
)

X_train_full = X_full_pd.iloc[train_idx]
X_test_full = X_full_pd.iloc[test_idx]
X_train_num = X_numeric[train_idx]
X_test_num = X_numeric[test_idx]
y_train = y[train_idx]
y_test = y[test_idx]

split_summary = pl.DataFrame(
    {
        "set": ["train", "test"],
        "rows": [len(y_train), len(y_test)],
        "default_rate": [round(y_train.mean(), 4), round(y_test.mean(), 4)],
    }
)
```

In [ ]:
```
split_summary
```

Search...

set str	rows i64	default_rate f64
train	89,521	0.0544
test	22,381	0.0544

2 rows, 3 columns

No selection

3. Evaluation metrics

In [ ]:
```
def compute_metrics(y_true, y_prob):
    return {
        "roc_auc": round(roc_auc_score(y_true, y_prob), 4),
        "pr_auc": round(average_precision_score(y_true, y_prob), 4),
        "log_loss": round(log_loss(y_true, y_prob), 4),
        "brier_score": round(brier_score_loss(y_true, y_prob), 4),
    }

def compute_ece(y_true, y_prob, n_bins=10):
    bin_edges = np.linspace(0, 1, n_bins + 1)
    ece = 0.0
    for i, (lo, hi) in enumerate(zip(bin_edges[:-1], bin_edges[1:])):
        mask = (y_prob >= lo) & (y_prob <= hi) if i == n_bins - 1 else (y_pr
        if mask.sum() == 0:
            continue
        avg_predicted = y_prob[mask].mean()
        avg_actual = y_true[mask].mean()
        ece += mask.sum() * abs(avg_actual - avg_predicted)
    return round(ece / len(y_true), 4)
```

4. Logistic regression (numeric features only)

In [ ]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_num)
X_test_scaled = scaler.transform(X_test_num)

lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)

lr_probs = lr_model.predict_proba(X_test_scaled)[:, 1]
lr_metrics = {"model": "logistic_regression", **compute_metrics(y_test, lr_p
```

In [ ]:
```
lr_metrics
```

5. XGBoost with default hyperparameters

In [ ]:
```
scale_pos_weight = float(np.sum(y_train == 0) / np.sum(y_train == 1))

xgb_default_model = xgb.XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale_pos_weight,
    enable_categorical=True,
    tree_method="hist",
    random_state=42,
    eval_metric="logloss",
)
xgb_default_model.fit(X_train_full, y_train)

xgb_default_probs = xgb_default_model.predict_proba(X_test_full)[:, 1]
xgb_default_metrics = {
    "model": "xgboost_default",
    **compute_metrics(y_test, xgb_default_probs),
}
```

In [ ]:
```
xgb_default_metrics
```

6. XGBoost with Optuna tuning

In [ ]:
```
def optuna_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 100, 800),
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 20),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-8, 10.0, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-8, 10.0, log=True
```

```
        "scale_pos_weight": scale_pos_weight,
        "enable_categorical": True,
        "tree_method": "hist",
        "random_state": 42,
        "eval_metric": "logloss",
    }
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = []
    for fold_train_idx, fold_val_idx in cv.split(X_train_full, y_train):
        model = xgb.XGBClassifier(**params)
        model.fit(X_train_full.iloc[fold_train_idx], y_train[fold_train_idx]
        probs = model.predict_proba(X_train_full.iloc[fold_val_idx])[:, 1]
        scores.append(roc_auc_score(y_train[fold_val_idx], probs))
    return np.mean(scores)

study = optuna.create_study(direction="maximize")
study.optimize(optuna_objective, n_trials=50)
best_params = study.best_params
```

In [ ]:
```
best_params
```

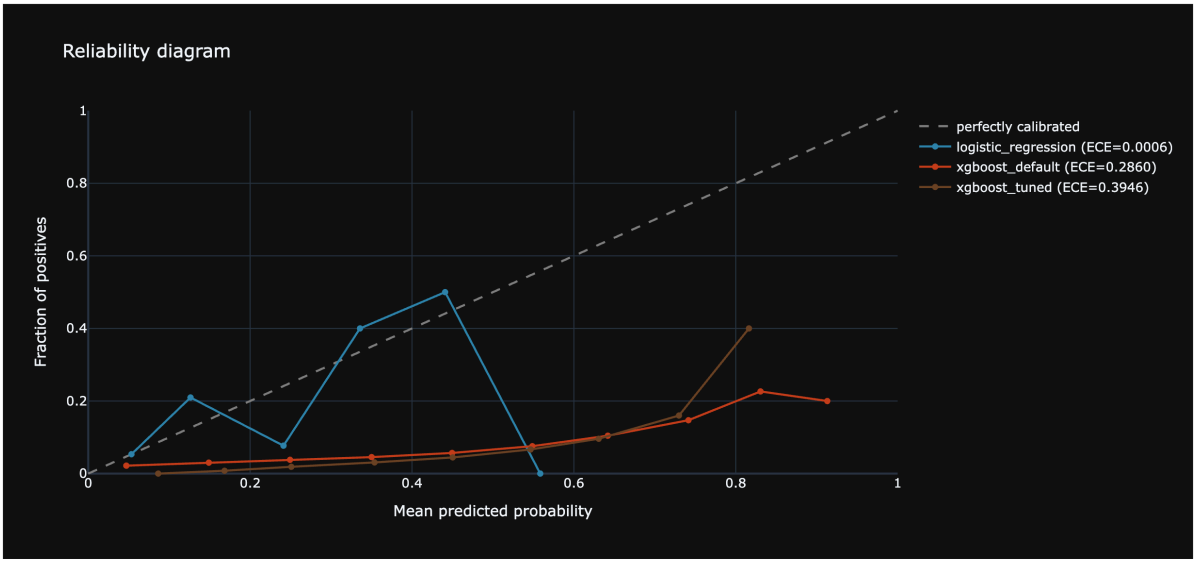
In [ ]:
```
xgb_tuned_model = xgb.XGBClassifier(
    **best_params,
    scale_pos_weight=scale_pos_weight,
    enable_categorical=True,
    tree_method="hist",
    random_state=42,
    eval_metric="logloss",
)
xgb_tuned_model.fit(X_train_full, y_train)

xgb_tuned_probs = xgb_tuned_model.predict_proba(X_test_full)[:, 1]
xgb_tuned_metrics = {
    "model": "xgboost_tuned",
    **compute_metrics(y_test, xgb_tuned_probs),
}
```

In [ ]:
```
xgb_tuned_metrics
```

7. Calibration assessment

In [ ]:
```
_COLORS = {
    "logistic_regression": "#2E86AB",
    "xgboost_default": "#C73E1D",
    "xgboost_tuned": "#6B4226",
}

calibration_fig = go.Figure()
calibration_fig.add_trace(
    go.Scatter(
        x=[0, 1],
        y=[0, 1],
        mode="lines",
```

```
        line={"dash": "dash", "color": "gray"},
        name="perfectly calibrated",
    )
)

ece_results = {}
for _name, _probs, _color in [
    ("logistic_regression", lr_probs, _COLORS["logistic_regression"]),
    ("xgboost_default", xgb_default_probs, _COLORS["xgboost_default"]),
    ("xgboost_tuned", xgb_tuned_probs, _COLORS["xgboost_tuned"]),
]:
    _fraction_pos, _mean_predicted = calibration_curve(
        y_test, _probs, n_bins=10, strategy="uniform"
    )
    ece_results[_name] = compute_ece(y_test, _probs)
    calibration_fig.add_trace(
        go.Scatter(
            x=_mean_predicted,
            y=_fraction_pos,
            mode="lines+markers",
            name=f"{_name} (ECE={ece_results[_name]:.4f})",
            line={"color": _color},
        )
    )

calibration_fig.update_layout(
    title="Reliability diagram",
    xaxis_title="Mean predicted probability",
    yaxis_title="Fraction of positives",
    height=520,
    xaxis={"range": [0, 1]},
    yaxis={"range": [0, 1]},
)

mo.ui.plotly(calibration_fig)
```



In [ ]:
```
ece_results
```

Isotonic calibration on tuned XGBoost

In [ ]:
```
calibrated_model = CalibratedClassifierCV(xgb_tuned_model, method="isotonic"
calibrated_model.fit(X_train_full, y_train)

calibrated_probs = calibrated_model.predict_proba(X_test_full)[:, 1]
calibrated_metrics = {
    "model": "xgboost_tuned_calibrated",
    **compute_metrics(y_test, calibrated_probs),
}
calibrated_ece = compute_ece(y_test, calibrated_probs)
```

In [ ]:
```
{**calibrated_metrics, "ece": calibrated_ece}
```

8. Model comparison

In [ ]:
```
all_metrics = [
    lr_metrics,
    xgb_default_metrics,
    xgb_tuned_metrics,
    calibrated_metrics,
]
comparison_df = pl.DataFrame(all_metrics).with_columns(
    pl.Series(
        "ece",
        [
            ece_results.get("logistic_regression"),
            ece_results.get("xgboost_default"),
            ece_results.get("xgboost_tuned"),
            calibrated_ece,
        ],
    )
)
```

In [ ]:
```
comparison_df
```

Search...

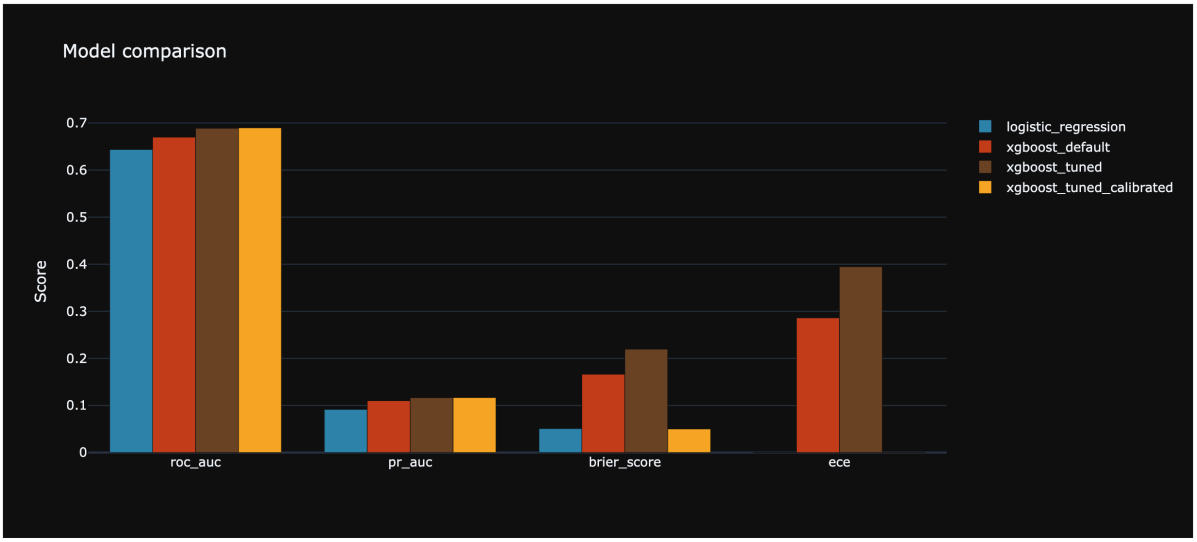
model str	roc_auc f64	pr_auc f64	log_loss f64	brier_score f64	ece f64
logistic_regression	0.6436	0.0916	0.2050	0.0508	0.0006
xgboost_default	0.6699	0.1100	0.4945	0.1663	0.2860
xgboost_tuned	0.6887	0.1166	0.6267	0.2196	0.3946
xgboost_tuned_calibrated	0.6897	0.1167	0.1991	0.0500	0.0004

4 rows, 6 columns

No selection

In [ ]:
```
_plot_metrics = ["roc_auc", "pr_auc", "brier_score", "ece"]
_colors = ["#2E86AB", "#C73E1D", "#6B4226", "#F5A623"]
_models = comparison_df["model"].to_list()

comparison_fig = go.Figure()
for _model, _color in zip(_models, _colors):
    _row = comparison_df.filter(pl.col("model") == _model)
    comparison_fig.add_trace(
```

```
        go.Bar(
            x=_plot_metrics,
            y=[_row[m].item() for m in _plot_metrics],
            name=_model,
            marker_color=_color,
        )
    )
comparison_fig.update_layout(
    title="Model comparison",
    barmode="group",
    yaxis_title="Score",
    height=500,
)
mo.ui.plotly(comparison_fig)
```



Xgboost only slightly improves on a simple logistic regression; moreover, we need to calibrate it in order to get a comparable Brier score and calibration score.

Statistical metrics are not enough to decide on a model to take to production; the bottom line is not some abstract number, but the actual business impact.

For this, we will implement a separate portfolio analysis where we take the models to work and simulate their performance on the whole history of loans, at various risk thresholds.

In order to facilitate that, we save the best XGBoost parameters to a JSON file that can be easily loaded in the next notebook.

In [ ]:
```
PARAMS_PATH = PROJECT_ROOT / "data" / "models"
PARAMS_PATH.mkdir(parents=True, exist_ok=True)

xgb_config = {
    **best_params,
    "scale_pos_weight": scale_pos_weight,
    "enable_categorical": True,
    "tree_method": "hist",
```

```
    "random_state": 42,
    "eval_metric": "logloss",
}

_output_path = PARAMS_PATH / "best_xgb_params.json"
_output_path.write_text(json.dumps(xgb_config, indent=2))
mo.md(f"Best XGBoost params saved to `{_output_path.relative_to(PROJECT_ROOT
```

Best XGBoost params saved to `data/models/best_xgb_params.json`

In [ ]: